

lps

COPPE/Poli/UFRJ

Laboratório de Processamento de Sinais

Inteligência Computacional. Inovação

Regularização

CPE727 - Aprendizado Profundo

Matheus Sotero

Programa de Engenharia Elétrica (PEE/COPPE)

22 de outubro de 2026

Conteúdo da Apresentação

1. Introdução
2. Regularização por Penalidade de Norma
3. Regularização por Reprojeção
4. Regularização por Parada Antecipada
5. Regularização por Aprendizado Multitarefa
6. Regularização por Vinculação/Compartilhamento de Parâmetros
7. Regularização por Representações Esparsas
8. Regularização por Injeção de Ruídos
9. Regularização *Dropout*

Introdução

O problema da generalização

Generalização em ML

O objetivo fundamental de qualquer modelo não é apenas minimizar o erro de treinamento, mas sim garantir um baixo **erro de generalização** (bom desempenho em dados novos e não vistos).

Subajuste (*Underfitting*)

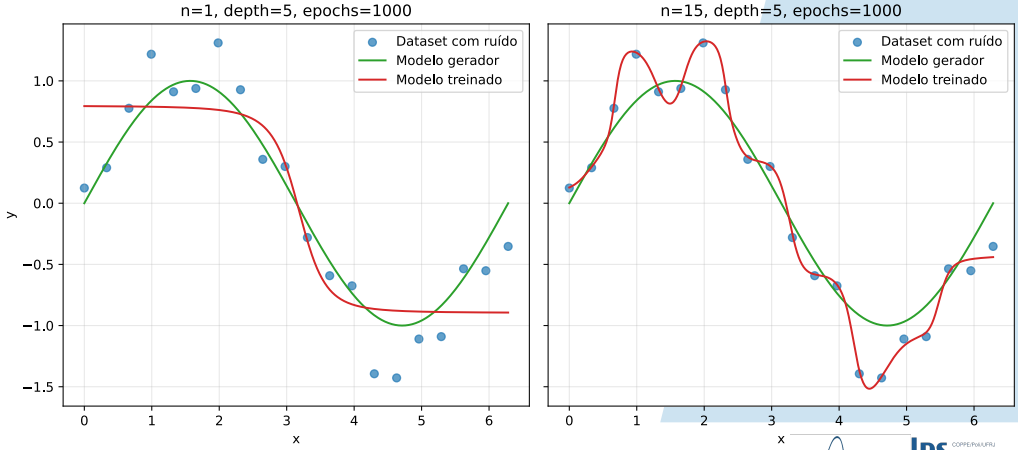
- Ocorre quando o modelo é simples demais e não se aproxima do processo real gerador de dados.
- Erro elevado tanto no treino quanto no teste.
- Dominado pelo **alto viés** (*bias*).

Sobreajuste (*Overfitting*)

- Ocorre quando o modelo memoriza o ruído do treino.
- Erro de treino baixo, mas erro de teste elevado.
- Dominado pela **alta variância** (*variance*).

Underfitting X Overfitting

Exemplo inspirado em [1], com ruído de alta variância ($\sigma = 0.25$)



Underfitting X Overfitting

A tabela abaixo mostra o MSE obtido sobre os conjuntos de treinamento e teste para cada uma das redes do exemplo

largura da rede	Treinamento	Teste
$n = 1$	0.129	0.167
$n = 15$	0.001	0.058

Resumo do problema

Não conseguimos partir de uma rede muito simples e fazer com que ela represente sistemas complexos. Mas para evitar o *overfitting* precisamos fazer com que as redes profundas a serem treinadas consigam:

- **Diminuir a alta variância:** Reduzir o erro de predição sobre o **conjunto de teste** (evitar a memorização do ruído).
- **Sem aumentar significativamente o viés:** Manter o erro de predição sobre o **conjunto de treinamento** baixo (preservar a capacidade do modelo).

Solução

A solução para este impasse é a **Regularização**.

Definição formal

Definição de regularização, segundo [2]

”Qualquer modificação feita a um algoritmo de aprendizado de máquina com o objetivo de reduzir o seu erro de generalização (teste), mas não o seu erro de treinamento.”

O Paradigma Moderno

- Em vez de buscar o tamanho “exato” da rede, a prática padrão em Deep Learning é:

Modelo de Alta Capacidade + Regularização Apropriada [2]

Regularização por Penalidade de Norma

Formulação geral

É uma família de métodos de regularização que buscam limitar a explosão dos parâmetros penalizando a sua norma diretamente na função objetivo.

Formulação geral da função de custo regularizada

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta})$$

- $\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y})$**:** Função de objetivo final a ser minimizada.
- $J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y})$**:** Função de custo original sobre os dados de treinamento.
- $\Omega(\boldsymbol{\theta})$**:** Função de penalidade sobre a norma dos parâmetros.
- $\alpha \in [0, \infty)$**:** Hiperparâmetro que pondera o impacto da regularização:

Observação importante

Em uma rede neural, os parâmetros penalizados são, em geral, apenas os pesos ($\boldsymbol{w}$) de cada camada.

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha \Omega(\boldsymbol{\theta})$$

$$\Omega(\boldsymbol{w})$$

Pesos (*Weights*) vs. Vieses (*Biases*)

"Em redes neurais, tipicamente penalizam-se apenas os **pesos (w)** de cada camada. Os **vieses (b)** não são regularizados pois eles não alteram de forma significativa a variância final do modelo" [2].

Regularização L^2 (*Weight Decay* ou Regularização Ridge)

Formulação da função de custo L^2

Adiciona uma penalidade proporcional ao quadrado da norma 2 dos pesos:

$$\Omega(\boldsymbol{w}) = \frac{1}{2}||\boldsymbol{w}||_2^2 \qquad \longrightarrow \qquad \tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) + \frac{\alpha}{2}\boldsymbol{w}^T\boldsymbol{w}$$

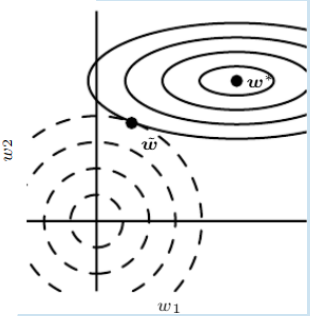
Para minimizar esse custo é usada a seguinte formulação na atualização dos parâmetros durante o treinamento:

$$\boldsymbol{w} \quad \leftarrow \quad (1 - \epsilon\alpha)\boldsymbol{w} - \epsilon\nabla_{\boldsymbol{w}} J(\boldsymbol{w}; \boldsymbol{X}, y)$$

A solução regularizada $\tilde{\boldsymbol{w}}$ busca um ponto de compromisso entre minimizar o erro de treino e manter a norma dos pesos pequena.

Mecanismo de funcionamento da Regularização L^2

- **Eixo w_1 (Baixa curvatura):**
  - O custo J varia muito pouco na horizontal ao se afastar de w^*.
  - A penalidade domina e **puxa w_1 para perto de zero**.
- **Eixo w_2 (Alta curvatura):**
  - J é altamente sensível a deslocamentos na vertical.
  - O custo domina e **afeta muito pouco a posição de w_2** ($\tilde{w}_2 \approx w_2^*$).



Curvas de nível (superfícies Hessianas) das funções:
$J(\boldsymbol{w})$ - linha sólida
$\Omega(\boldsymbol{w})$ - linha pontilhada

Conclusão: A regularização L^2 preserva apenas os parâmetros essenciais para a redução da perda e descarta componentes irrelevantes.

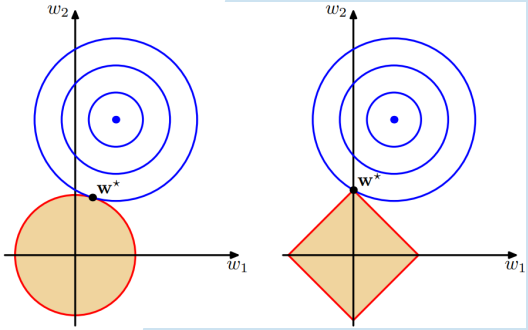
Regularização L^1 (Regularização Lasso)

Formulação da Função de Custo L^1

Adiciona uma penalidade proporcional à soma dos valores absolutos dos pesos:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) + \alpha \|\boldsymbol{w}\|_1$$

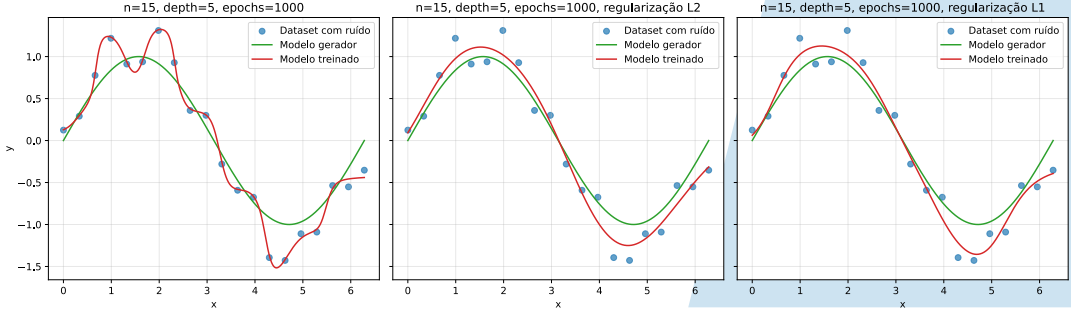
O contorno do L^1 possui vértices (*corners*) nos eixos (na figura de exemplo, $w_1 = 0$ ou $w_2 = 0$), favorecendo interseções com a função objetivo diretamente sobre os eixos (esparsidade) [2]



Exemplo numérico de aplicação

Mesmo dataset do exemplo anterior, inspirado em [1], com 5 camadas de profundidade + 15 neurônios por camada:

Sem regularização **Regularização L^2** **Regularização L^1**

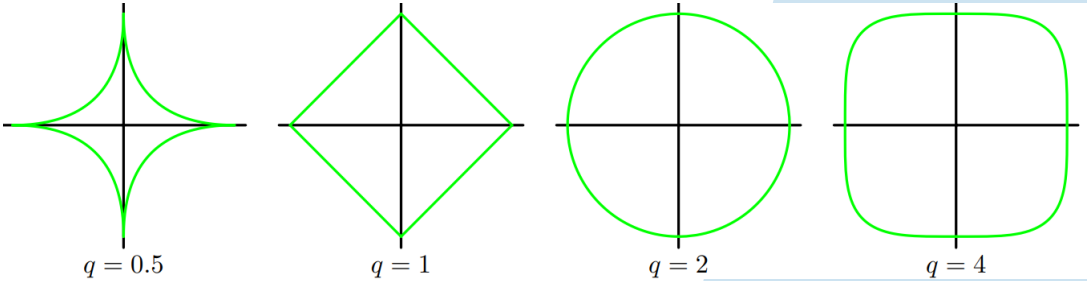


Outros contornos de regularização

Podemos generalizar a função $\Omega(\boldsymbol{w})$ de penalização da norma com o formato

$$\Omega(\boldsymbol{w}) = ||\boldsymbol{w}||_q \ ,$$

obtendo outras curvas de nível.



Regularização por Reprojeção

Restrições explícitas

Penalidade Suave vs. Restrição Rígida

Em vez de somar a penalidade $\alpha\Omega(\boldsymbol{\theta})$ à função de custo, podemos limitar os pesos a uma região de tamanho k:

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) \quad \text{sujeito a} \quad \Omega(\boldsymbol{\theta}) \leq k$$

Formulação via multiplicadores de Lagrange:

$$\mathcal{L}(\boldsymbol{\theta}, \alpha) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha(\Omega(\boldsymbol{\theta}) - k)$$

- **Equivalência Matemática:** Para cada limite k, existe um multiplicador α^* ótimo que torna a otimização restrita equivalente à penalidade suave [2].

Regularização por Reprojeção

Mudanças do algoritmo de regularização por reprojeção

Para cada iteração t com taxa de aprendizado ϵ e limite de norma k:

1. **Cálculo do gradiente em J:**

$$\boldsymbol{\theta}' \leftarrow \boldsymbol{\theta}^{(t)} - \epsilon \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{(t)})$$

2. **Etapa de reprojeção (projeção na fronteira):**

$$\boldsymbol{\theta}^{(t+1)} \leftarrow \begin{cases} \boldsymbol{\theta}', & \text{se } \Omega(\boldsymbol{\theta}') \leq k \\ k \cdot \dfrac{\boldsymbol{\theta}'}{\Omega(\boldsymbol{\theta}')}, & \text{se } \Omega(\boldsymbol{\theta}') > k \end{cases}$$

Vantagens da reprojeção para aprendizado profundo

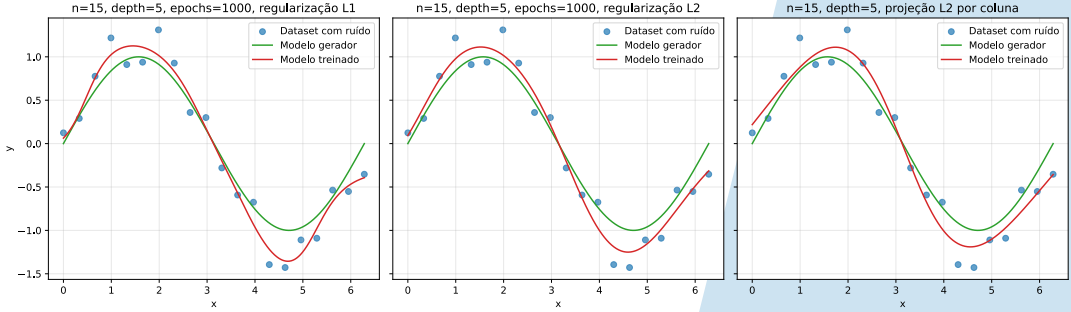
Porque usar restrições explícitas em vez de penalidades suaves?

A regularização por reprojeção impõe uma restrição rígida ($\Omega(\boldsymbol{\theta}) \leq k$), atuando apenas quando o modelo tenta violar o limite, permitindo otimização livre sem puxar os pesos internos em direção a zero.

- **Ajuste intuitivo de hiperparâmetros:** É mais simples e direto determinar o valor limite k da norma do que buscar o hiperparâmetro de penalidade α correspondente.
- **Estabilidade numérica com altas taxas de aprendizado:** Permite o uso de taxas de aprendizado (*learning rates*) elevadas para uma exploração mais rápida do espaço de busca [3].
  - Em particular, [3] sugere que a restrição restrita seja aplicada para cada coluna da matriz de pesos de forma separada.

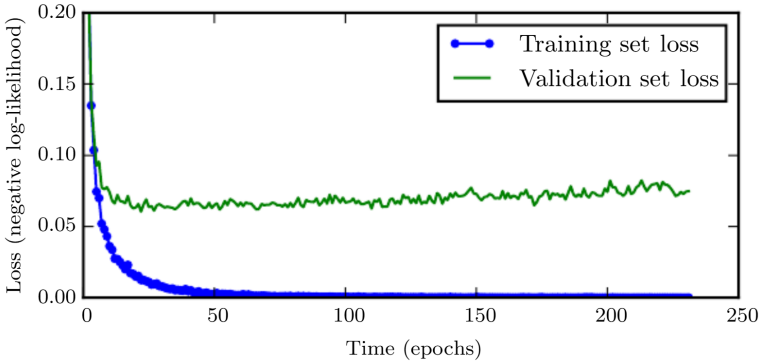
Exemplo de regularização por reprojeção

Regularização L^1 | **Regularização L^2** | **Reprojeção**



Regularização por Parada Antecipada

Curvas de erro de treinamento e validação



Comportamento das curvas de erro: Enquanto a perda sobre o treino cai continuamente, a erro de validação forma uma curva com ponto mínimo, subindo ao iniciar o *overfitting*

Parada Antecipada (*Early Stopping*)

O que é?

É uma estratégia de regularização que monitora periodicamente o erro no conjunto de validação e interrompe o treinamento quando o desempenho deixa de melhorar

- **Principal ideia:** Parar o avanço do gradiente antes do *overfitting*.

Meta-algoritmo da regularização por parada antecipada

Algorithm 1 Meta-algoritmo da parada antecipada (Early Stopping)

Require: Número de passos entre avaliações n, paciência ρ e parâmetros iniciais θ_0

Inicializar o modelo e registrar seus parâmetros como os melhores até então.
while não atingir a paciência ρ **do**
    Treinar o modelo por n passos e avaliar o erro de validação.
    if o erro de validação diminuir **then**
        Atualizar os melhores parâmetros e reiniciar a contagem de paciência.
    else
        Incrementar a contagem de paciência.
    end if
end while
return Melhores parâmetros encontrados θ^* e o número de passos do treinamento para alcançá-los τ.

Equivalência a L^2

Relação com a Regularização L^2

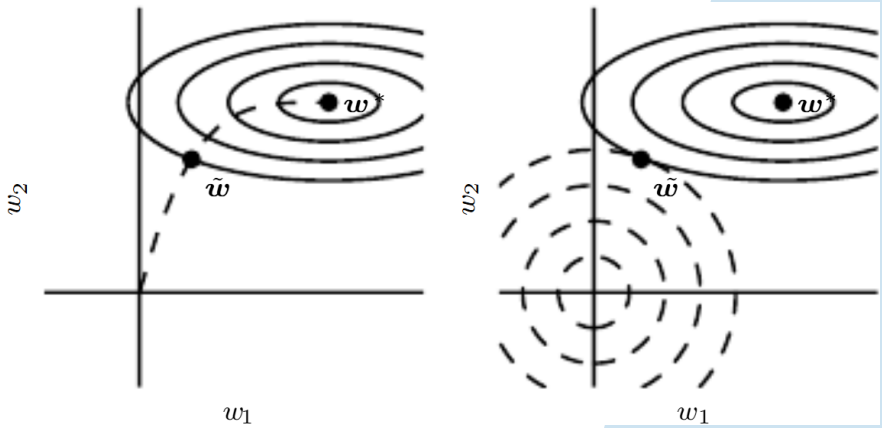
Sob uma aproximação quadrática e gradiente descendente com taxa ϵ, o número de iterações τ é inversamente proporcional ao parâmetro α da restrição suave:

$$\tau \approx \frac{1}{\epsilon\alpha} \quad \Longleftrightarrow \quad \alpha \approx \frac{1}{\tau\epsilon}$$

Vantagem prática para o aprendizado profundo

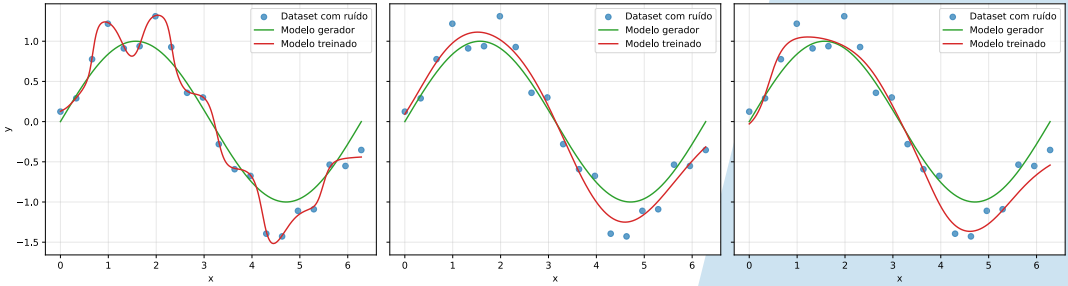
- **Restrição do espaço de busca:** Limita a trajetória em τ passos e impede que os parâmetros se afastem excessivamente da inicialização, fazendo com que não seja gastado recursos e tempo dando passos de treinamento desnecessários.

Equivalência a L^2



Exemplo de aplicação

Sem regularização **Regularização L^2** **Parada Antecipada**



Regularização por Aprendizado Multitarefa

Ideia geral

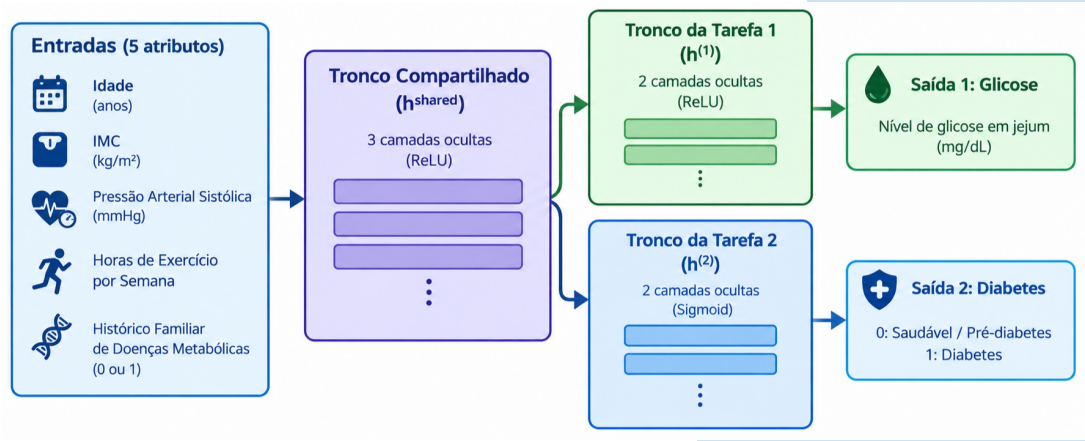
O que é?

É uma estratégia para melhorar a generalização treinando um único modelo para resolver múltiplas tarefas relacionadas simultaneamente, acumulando os dados de todas as tarefas.

- **Divisão da arquitetura em duas partes:**
  - **Parâmetros Compartilhados:** Camadas superiores que processam a entrada comum x e geram uma representação intermediária $h^{(\text{shared})}$.
  - **Parâmetros Específicos:** Camadas inferiores dedicadas a prever a saída $y^{(i)}$ de cada tarefa individual.
- **Premissa do método:** Assume-se que existe um conjunto comum de fatores ocultos que explicam as variações nos dados entre as diferentes tarefas [2].

Exemplo de arquitetura

Exemplo numérico disponível em: FALTA COLOCAR LINK DO REPOSITÓRIO



Formulação

Formulação do custo combinado

A otimização minimiza a soma ponderada das funções de perda de todas as T tarefas sobre os parâmetros compartilhados ($\boldsymbol{\theta}_{\text{shared}}$) e específicos ($\boldsymbol{\theta}^{(i)}$):

$$\tilde{J}\left(\boldsymbol{\theta}_{\text{shared}}, \boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(T)}\right) = \sum_{i=1}^{T} \alpha^{(i)} J^{(i)}\left(\boldsymbol{\theta}_{\text{shared}}, \boldsymbol{\theta}^{(i)}\right)$$

Conclusão: O compartilhamento de parâmetros atua como a **restrição suave** [2].

- Ao acumular a força estatística das amostras de todas as tarefas, o modelo é **pressionado a aprender recursos gerais e robustos** nas camadas compartilhadas, descartando o ruído específico de tarefas individuais.

Regularização por Vinculação/Compartilhamento de Parâmetros

Ideia geral

No que consiste?

Em vez de penalizar o afastamento em relação a um ponto fixo (como a origem no L^2), expressa-se o conhecimento prévio de que determinados parâmetros devem ser semelhantes ou iguais entre si [2].

- **Vinculação de Parâmetros (*Parameter Tying*):**
  - É uma restrição suave, via penalidade de norma, sobre a diferença entre dois conjuntos de parâmetros de modelos ou componentes distintos.
- **Compartilhamento de Parâmetros (*Parameter Sharing*):**
  - É uma restrição rígida que força conjuntos de parâmetros a serem estritamente iguais, redefinindo a arquitetura para reutilizar um conjunto único de pesos.
  - Reduz significativamente a quantidade de parâmetros livres e a memória necessária para armazená-los.

Formulação

Formulação da proximidade dos parâmetros

Para dois modelos ou componentes A e B com pesos $\boldsymbol{w}^{(A)}$ e $\boldsymbol{w}^{(B)}$:

- **Vinculação (*Parameter Tying*):** $\Omega(\boldsymbol{w}^{(A)}, \boldsymbol{w}^{(B)}) = \|\boldsymbol{w}^{(A)} - \boldsymbol{w}^{(B)}\|_2^2$
- **Compartilhamento (*Parameter Sharing*):** Restrição rígida $\boldsymbol{w}^{(A)} = \boldsymbol{w}^{(B)}$

Uso consagrado em DeepNN: Redes Convolucionais (CNNs)

- **Invariância por transladação:** O mesmo detector de atributos (filtro/kernel) é reutilizado em todas as posições espaciais da imagem.
- **Drástica redução de parâmetros:** Permite construir redes muito maiores sem necessitar de um aumento proporcional no volume de dados.

Regularização por Representações Esparsas

Esparsidade de parâmetros vs. de representação:

- **De Parâmetros:** A maioria dos pesos $\boldsymbol{w}$ assume valor zero (induzida por norma L^1 nos parâmetros).
- **De Representação:** O vetor de ativação $\boldsymbol{h}$ contém a maioria de suas entradas nulas para uma determinada amostra $\boldsymbol{x}$.

$$\underset{\boldsymbol{y} \in \mathbb{R}^m}{\begin{bmatrix} 18 \\ 5 \\ 15 \\ -9 \\ -3 \end{bmatrix}} = \underset{\boldsymbol{A} \in \mathbb{R}^{m \times n}}{\begin{bmatrix} 4 & 0 & 0 & -2 & 0 & 0 \\ 0 & 0 & -1 & 0 & 3 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 & -4 \\ 1 & 0 & 0 & 0 & -5 & 0 \end{bmatrix}} \underset{\boldsymbol{x} \in \mathbb{R}^n}{\begin{bmatrix} 2 \\ 3 \\ -2 \\ -5 \\ 1 \\ 4 \end{bmatrix}} \tag{7.46}$$

$$\underset{\boldsymbol{y} \in \mathbb{R}^m}{\begin{bmatrix} -14 \\ 1 \\ 19 \\ 2 \\ 23 \end{bmatrix}} = \underset{\boldsymbol{B} \in \mathbb{R}^{m \times n}}{\begin{bmatrix} 3 & -1 & 2 & -5 & 4 & 1 \\ 4 & 2 & -3 & -1 & 1 & 3 \\ -1 & 5 & 4 & 2 & -3 & -2 \\ 3 & 1 & 2 & -3 & 0 & -3 \\ -5 & 4 & -2 & 2 & -5 & -1 \end{bmatrix}} \underset{\boldsymbol{h} \in \mathbb{R}^n}{\begin{bmatrix} 0 \\ 2 \\ 0 \\ 0 \\ -3 \\ 0 \end{bmatrix}} \tag{7.47}$$

Abordagem para obter essa representação esparsa

A ideia é penalizar diretamente sobre os vetores de ativação $\boldsymbol{h}$

Formulação matemática

- **Penalidade Suave (Norma L^1 nas Ativações):**

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{h}), \quad \text{onde } \Omega(\boldsymbol{h}) = \|\boldsymbol{h}\|_1 = \sum_i |h_i|$$

- **Restrição Rígida (*Hard Constraint* / OMP):**

$$\min_{\boldsymbol{h}} \|\boldsymbol{x} - \boldsymbol{W}\boldsymbol{h}\|_2^2 \quad \text{sujeito a} \quad \|\boldsymbol{h}\|_0 < k$$

onde $\|\boldsymbol{h}\|_0$ representa o número de elementos não-nulos em $\boldsymbol{h}$.

Vantagem

Principal vantagem para Aprendizado Profundo

- **Seleção Adaptativa de Atributos:** Força o modelo a descrever cada entrada x utilizando apenas uma combinação reduzida de características ativas.

Regularização por Injeção de Ruídos

Ideia geral

O que é?

A injeção de ruído consiste em injetar um ruído estocástico em alguma parte do modelo de modo que force a rede a aprender as invariâncias do sistema .

- **Locais de aplicação de ruído na rede:**
  - **Nas entradas:** Atuando como uma forma sintética de aumento de dados (*Dataset Augmentation*), suaviza a função aprendida e cria resistência a pequenas perturbações.
  - **Nas camadas ocultas de ativação:** Promove o aumento de dados em múltiplos níveis de abstração (base conceitual do *Dropout*).
  - **Nos pesos:** Atua como um regularizador Bayesiano, estabilizando a otimização em regiões de menor sensibilidade.

Injeção de ruído nas entradas

Formulação Matemática

Adiciona-se uma perturbação aleatória $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ diretamente ao dado de entrada, treinando o modelo com o conjunto original acrescido com $\tilde{\boldsymbol{x}} = \boldsymbol{x} + \boldsymbol{\epsilon}$.

Uso em DeepNNs

- **Indução de suavidade:** Obriga o modelo a produzir previsões estáveis diante de pequenas variações do mundo real. [2]
- **Aumento do dataset:** Aumenta o tamanho do dataset de treinamento, o que pode ser muito útil para datasets pequenos [2]

Injeção de ruído nos pesos

Formulação Matemática

Para um modelo $\hat{y}(\boldsymbol{x})$, injeta-se ruído $\boldsymbol{\epsilon}_W \sim \mathcal{N}(\mathbf{0}, \eta\mathbf{I})$ nos parâmetros. A minimização do custo perturbado $\tilde{J}_W = \mathbb{E}_{\boldsymbol{\epsilon}_W}\left[(\hat{y}_{\boldsymbol{\epsilon}_W}(\boldsymbol{x}) - y)^2\right]$ equivale a [2]:

$$\tilde{J}_W = J + \eta\,\mathbb{E}_{p(\boldsymbol{x},y)}\left[\|\nabla_W \hat{y}(\boldsymbol{x})\|^2\right]$$

Mecanismo e aplicações em aprendizado profundo

- **Busca por Mínimos Planos (*Flat Minima*):** Empurra os parâmetros para regiões do espaço onde pequenas variações nos pesos não alteram significativamente a saída.
- **Tratamento Bayesiano:** Permite uma interpretação probabilística dos pesos, sendo amplamente utilizado em RNNs [2]

Exemplo de aplicação para o dataset MNIST

Modelo	Treino	Teste
Sem regularização	97.67%	96.54%
Injeção de Ruídos na Entrada	97.64%	96.93%
Injeção de Ruídos nos Pesos	98.34%	97.35%

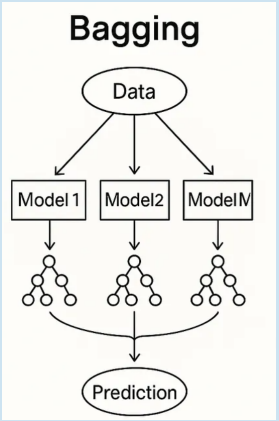


Regularização *Dropout*

Bagging e métodos de assembleia

Estratégia

O *bagging* tenta reduzir o erro de generalização treinando múltiplos modelos independentes e combinando suas previsões por votação ou média.



- **Redução da Variância:**
  - Erros não correlacionados se cancelam na média, garantindo que o conjunto funcione tão bem ou melhor que qualquer membro isolado.
  - **Limitação em Deep Learning:** Treinar dezenas de redes neurais profundas independentes é **computacionalmente impraticável**.

Dropout: Conceito

O que é o Dropout?

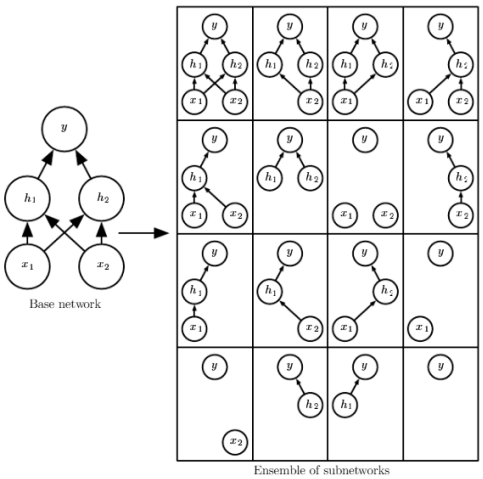
Técnica de regularização computacionalmente barata que torna o *bagging* prático para redes neurais profundas, treinando um número exponencial de sub-redes.

- **Sub-Redes com Compartilhamento de Parâmetros:**
  - A cada mini-batch, desativam-se aleatoriamente unidades de entrada e ocultas com probabilidade p.
  - Ao contrário do *bagging* tradicional, todas as sub-redes compartilham os mesmos parâmetros ($\boldsymbol{\theta}$), permitindo representar um número gigante de modelos com memória tratável.

Número exponencial de sub-redes

Conectividade de caminhos

- **Redes Pequenas:** Existe chance razoável de sortear sub-redes sem nenhum caminho ativo entre entrada e saída.
- **Redes Grandes:** Com camadas mais largas, a probabilidade de desconectar todos os caminhos é quase nula.
- **Garantia de Sinal:** A propagação do fluxo de informação é mantida durante todo o treinamento.



Formulação e regra de inferência

Formulação da Função Objetivo

Dado um vetor de máscara binária estocástica $\boldsymbol{\mu} \sim p(\boldsymbol{\mu})$, a otimização minimiza a esperança do custo sobre todas as sub-redes amostradas [5]:

$$\tilde{J}(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\mu} \sim p(\boldsymbol{\mu})} \left[J(\boldsymbol{\theta}, \boldsymbol{\mu}) \right]$$

Inferência: Regra de Escalonamento de Pesos

- Na fase de teste, utiliza-se a rede completa em um único *forward pass*.
- **Ajuste de Escala:** Multiplicam-se os pesos de saída da unidade pela probabilidade de inclusão p.

Exemplo de aplicação para o dataset MNIST

Modelo	Treino	Teste
Sem regularização	97.67%	96.54%
Injeção de Ruídos na Entrada	97.64%	96.93%
Injeção de Ruídos nos Pesos	98.34%	97.35%
Dropout	98.55%	97.15%



Referências I

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Information Science and Statistics, Springer, 1st ed., 2006.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
http://www.deeplearningbook.org.

[3] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *CoRR*, vol. abs/1207.0580, 2012.

Chegamos ao Fim
Obrigado pela sua Atenção